MODULE-13

DevOps AND CI/CD

DevOps

DevOps is a practice that combines development and operations to build, test and release software faster and more reliably.

It focuses on automation, collaboration and continuous delivery to improve software quality and speed.

- Helps teams release software faster with automation
- Reduces manual errors and improves reliability
- Enables continuous integration and deployment
- Improves collaboration between development and operations teams

Goals:

- Improve collaboration between development and operations teams.
- Automate software development and deployment processes.
- Deliver software faster with higher quality.
- Ensure continuous integration, testing, and delivery.

Benefits of DevOps:

- Faster software releases.
- Improved collaboration and communication.
- Higher software quality through continuous testing.
- Reduced deployment failures and quicker recovery.
- Increased efficiency through automation.

Stages of DevOps

1. **Plan Stage:**
   Defining project goals, requirements, and task breakdowns to align development and operations teams.
   - Work is broken down into tasks and user stories to ensure clarity and alignment.
   - Collab between dev team and ops team begins here.
   - Tools: Jira, Confluence, Azure Boards, Trello.

2. **Code Stage:**
   Writing, reviewing, and managing application source code and configurations using version control.
   - Code reviews and branching strategies help maintain code quality and stability.
   - Common Tools: Git, GitHub, GitLab, Bitbucket.
3. **Build Stage:**
   Automatically compiling and packaging code into deployable artifacts while resolving dependencies.
   - Build automation ensures faster feedback and reduces manual errors.
   - Common Tools: Jenkins, GitLab CI/CD, Maven, Gradle, Docker.
4. **Test Stage:**
   Running automated quality, security, and performance checks to identify bugs before release.
   - unit, integration, performance, and security testing.
   - Issues are identified early, that will reduce the cost and impact of failures.
   - Common Tools: Selenium, JUnit, TestNG, SonarQube, JMeter
5. **Release Stage:**
   Finalizing and documenting approved builds for deployment through version tagging and strategy planning.
   - Deployment strategies are planned to minimize risk during production roll out.
   - Common Tools**:** Git tags, Jenkins, GitLab CI/CD, ArgoCD.
6. **Deploy Stage:**
   Pushing the application into production environments using automated infrastructure and rollout strategies.
   - Supports strategies like **Blue-Green, Canary, and Rolling Updates** for minimal downtime.
   - Common Tools: Kubernetes, Helm, Ansible, Terraform.
7. **Operate and Monitor Stage:**
   Maintaining system health and gathering real-world performance data to drive continuous improvement.
   - Continuously monitor application performance, availability, and system health.
   - Logs, metrics, and alerts help detect and resolve issues quickly.
   - Monitoring and user feedback are used for continuous improvement.
   - Common Tools: Prometheus, Grafana, ELK Stack, Datadog, New Relic

INTRODUCTION TO CI/CD

CI/CD (Continuous Integration and Continuous Delivery/Deployment) is a modern software development practice that automates the process of building, testing, and releasing applications.

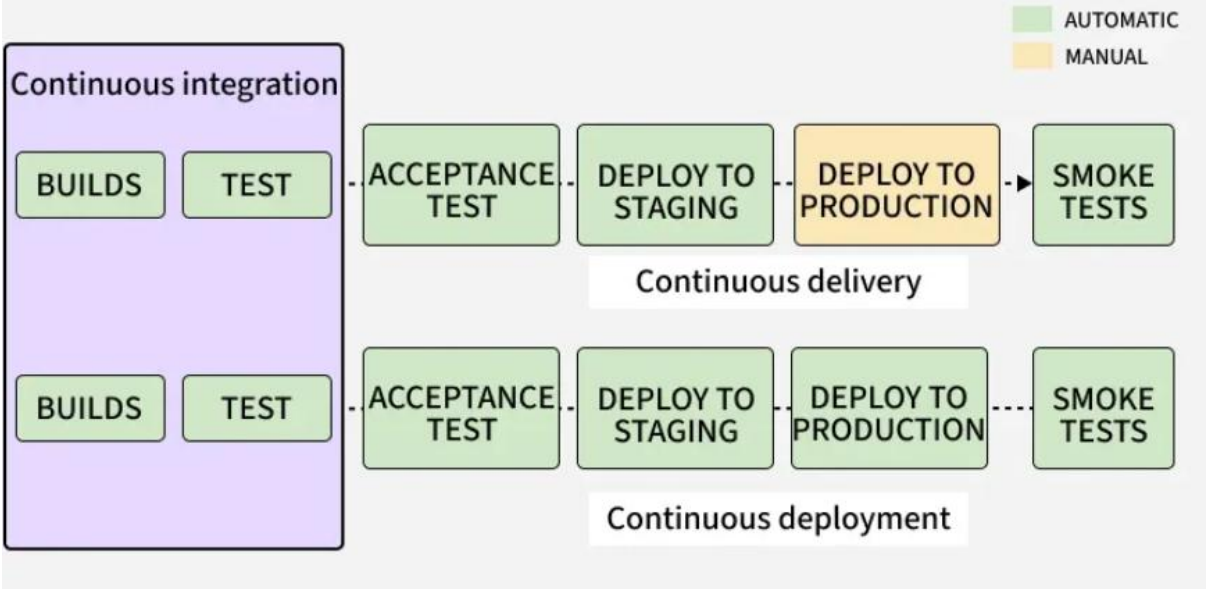
- Automates code integration, testing, and deployment workflows.
- Reduces manual effort while improving software quality.
- Smaller, frequent updates replace large risky releases.

Three Pillars of CI/CD

1. **Continuous Integration (CI):**
   Continuous Integration focuses on integrating code changes frequently to avoid conflicts and ensure code stability.
   - To prevent late code merging
   - More frequent code changes into the main branch
   - Each commit triggers an automated build and unit tests
   - If tests fail, the build is rejected and developers are notified immediately.

2. **Continuous Delivery (CD):**
   Continuous Delivery ensures that the application is always ready for release, with minimal manual effort.
   - Keep the codebase in a release-ready state at all times.
   - After CI passes, code is deployed to a staging or testing environment.
   - Integration, system, and performance tests are executed automatically.
   - Deployment to production is manual, typically triggered when needed.

3. **Continuous Deployment (CD):**
   Continuous Deployment takes automation a step further by removing manual intervention in releases.
   - Enable fully automated and faster production releases.
   - After all tests pass, code is automatically deployed to production.
   - End-to-end pipeline runs without human involvement.
   - Requires highly reliable and comprehensive automated testing.



Benefits of CI/CD:

- Faster and more frequent software releases.
- Early detection of bugs through automated testing.
- Reduces manual effort and deployment errors.
- Improves code quality and reliability.
- Enables faster feedback and easier collaboration.
- Ensures consistent and reliable deployments.

Common CI/CD Tools:

1. **Jenkins:** Open-source automation server widely used for building CI/CD pipelines.
2. **GitHub Actions:** CI/CD tool integrated with GitHub repositories.
3. **GitLab CI/CD:** Built-in CI/CD solution within GitLab.
4. **Concourse:** Open-source tool focused on pipeline automation.
5. **GoCD:** Provides visualization and management of complex pipelines.
6. **Spinnaker:** Continuous delivery platform for multi-cloud deployments.
7. **Screwdriver:** Platform designed for scalable CI/CD workflows.